

Algoritmi e Strutture Dati–parte I

lez. 4

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

Insertion-Sort(A)

1.	For $j \leftarrow 2$ to $\text{Length}(A)$	$c_1 n$
2.	$\text{Key} \leftarrow A[j]$	$c_2 (n - 1)$
3.	$i \leftarrow j - 1$	$c_3 (n - 1)$
4.	While ($i > 0$ and $A[i] > \text{key}$)	c_4
5.	Do $A[i+1] \leftarrow A[i]$	c_5
6.	$i \leftarrow i - 1$	c_6
7.	$A[i+1] \leftarrow \text{key}$	$c_7 (n - 1)$

**Tempo di
Esecuzione**

Insertion Sort: Caso Favorevole

- Il caso Favorevole si ha quando **l' input e` gia` ordinato**. In tal caso il loop non viene eseguito.
- Il tempo di esecuzione $T(n)$ e`
$$T(n) = c_1n + (c_2 + c_3 + c_7)(n - 1)$$

Ossia **$T(n)$ è una funzione lineare di n .**

Insertion Sort: Caso Peggior

- L' input è ordinato in modo decrescente. Ad es. $A = (17,15,11,8,7,5,4,2,1)$
- In questo caso il tempo di esecuzione $T(n)$ è dato da:
$$T(n) = P(n) + c_4 \sum_{j=2}^n j + (c_5 + c_6) \sum_{j=2}^n (j - 1)$$
- Ma $\sum_{j=2}^n j + 1 - 1 = \sum_{j=1}^n j - 1 = \frac{n(n+1)}{2} - 1$
- $\sum_{j=2}^n (j - 1) = \sum_{j=2}^n j - \sum_{j=2}^n 1 = \frac{n(n+1)}{2} - 1 - (n - 1) = \frac{n(n-1)}{2}$
- $T(n)$ è un polinomio di grado 2 di n ossia la complessità è $\Theta(n^2)$.

Insertion Sort: Caso Medio

- L' input è ordinato in modo decrescente. Ad es. $A = (17, 15, 11, 8, 7, 5, 4, 2, 1)$
- In questo caso il tempo di esecuzione $T(n)$ è dato da:
$$T(n) = P(n) + c_4 \sum_{j=2}^n j/2 + (c_5 + c_6) \sum_{j=2}^n (j-1)/2$$
- Ma $\frac{1}{2} \sum_{j=2}^n j + 1 - 1 = \sum_{j=1}^n j - 1 = \frac{1}{2} \frac{n(n+1)}{2} - 1$
- $\frac{1}{2} \sum_{j=2}^n (j-1) = \sum_{j=2}^n j - \sum_{j=2}^n 1 = \frac{1}{2} \left(\frac{n(n+1)}{2} - 1 - (n-1) \right) = \frac{1}{2} \frac{n(n-1)}{2}$
- $T(n)$ è un polinomio di grado 2 di n ossia la complessità è $\Theta(n^2)$.

Sommatorie

- La somma dei primi n numeri interi è data da:

$$1 + 2 + 3 + \cdots = \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Insertion Sort: Caso medio

- Il caso medio è spesso cattivo quanto il peggiore. Supponiamo di aver scelto a caso n numeri e di applicare l'insertion sort. Quanto tempo impiegherà per determinare dove inserire $A[j]$ nel sottoarray $A[1 \dots j - 1]$?
- In media metà degli elementi saranno più piccoli di $A[j]$, mentre gli altri elementi sono più grandi. Quindi verifichiamo metà del sottoarray $A[1 \dots j - 1]$, quindi $j/2$.
- Il **tempo di esecuzione del caso medio per l'insertion sort risulta dunque una funzione quadratica della dimensione dell'input**, come nel caso peggiore.

Paradigma Divide Et Impera

Divide et impera: Orazi e Curiazi



Divide et Impera: la leggenda degli Orazi e Curiazi

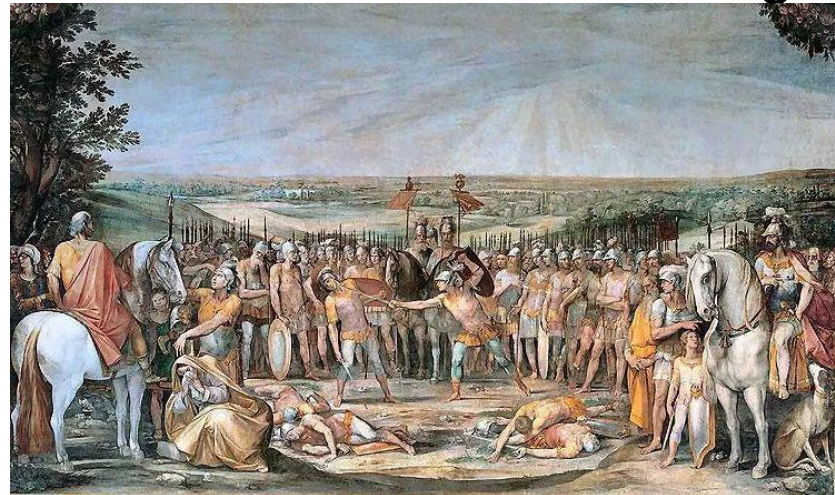
- Secondo la versione riportata da Tito Livio, durante il regno di Tullo Ostilio (VII secolo a.C.) Roma ed Albalonga entrarono in guerra, affrontandosi con gli eserciti schierati lungo le *Fossae Cluiliae*.
- Ma Roma ed Albalonga condividevano attraverso il mito di Romolo una sacra discendenza che rendeva empia questa guerra, perciò i rispettivi sovrani decisero di affidare a due gruppi di rappresentanti le sorti del conflitto fra le due città, evitando ulteriori spargimenti di sangue.

(cont.)

- Furono scelti per Roma gli Orazi, tre fratelli figli di Publio Orazio, e per Albalonga i tre gemelli Curiazi, che si sarebbero affrontati a duello alla spada.
- Iniziato il combattimento, quasi subito due Orazi furono uccisi, mentre due dei Curiazi riportarono solo lievi ferite; il terzo Orazio, che non avrebbe potuto affrontare da solo tre nemici, trovandosi in difficoltà pensò di ricorrere all'astuzia e finse di scappare verso Roma.
- Come aveva previsto, i tre Curiazi lo inseguirono, ma nel correre si distanziarono fra loro, perché feriti in modo differente inseguivano a velocità differenti.

(cont.)

- Per primo fu raggiunto dal Curiazio che non era stato ferito e, voltandosi a sorpresa, lo trafisse. Riprese a correre e fu poi raggiunto da ciascuno degli altri due Curiazi, che però, essendo feriti, si stancarono notevolmente e gli fu facile, uno alla volta, ucciderli. La vittoria dell'Orazio fu la vittoria di Roma, cui Albalonga si sottomise.



DIVIDE-et-Impera (Divide and Conquer)

La Tecnica di Programmazione DIVIDE-et-Impera ha i seguenti passi:

- **DIVIDE** (il problema viene suddiviso in sottoproblemi)
- **IMPERA** (i problemi vengono risolti anche ricorsivamente)
- **COMBINA** (le soluzioni dei sottoproblemi vengono combinate)

L' approccio DIVIDE-et-IMPERA è un approccio top-down

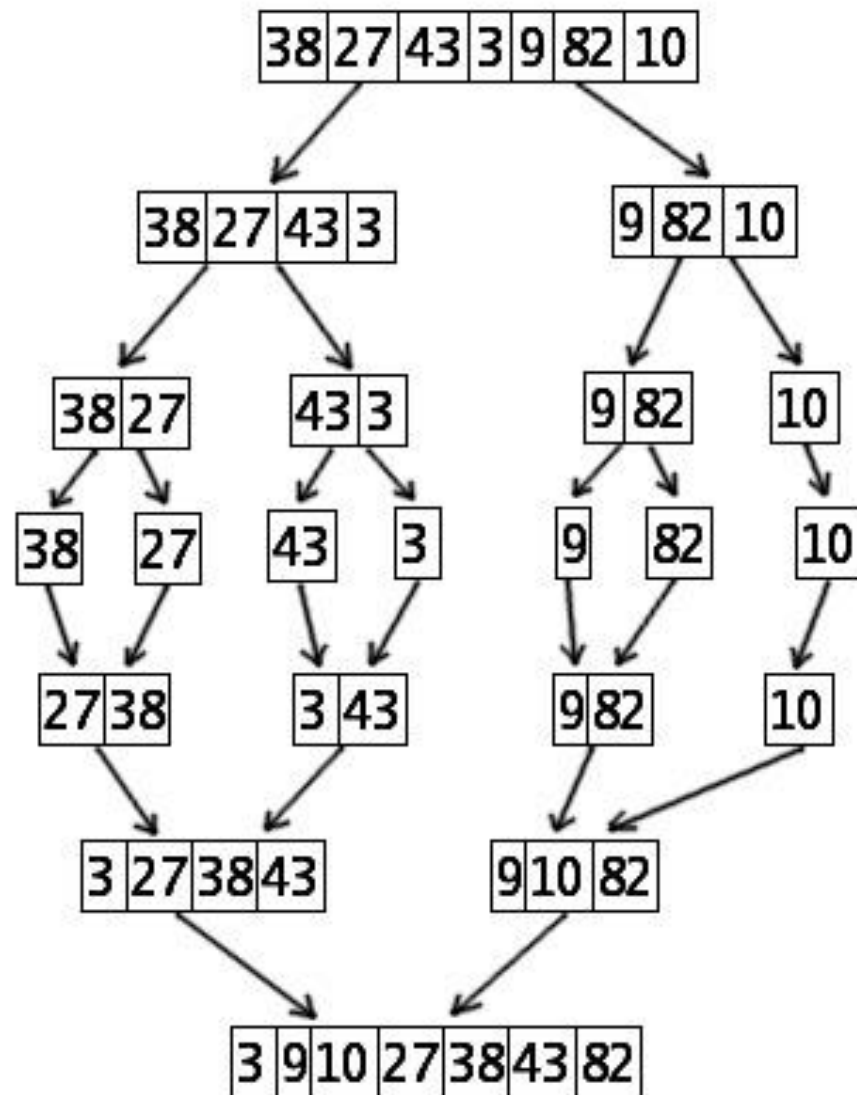
Merge-Sort

- Merge-Sort è un algoritmo di ordinamento con complessità $\Theta(n \log n)$.
- È un algoritmo di **ordinamento stabile**, cioè mantiene l'ordine iniziale (di input) di elementi uguali nell'array ordinato di output.
- Merge-Sort è un esempio del paradigma algoritmico divide-et-impera.
- È un esempio di comparison sort.
- Mergesort è stato inventato da **John von Neumann** nel 1945.

Merge Sort: Funzionamento

Concettualmente il funzionamento del Merge Sort è il seguente

- **Dividi l' array** disordinato in **due sottoliste** di (circa) metà lunghezza (passo **DIVIDE** in DIVIDE-ET-IMPERA).
- **Ordina**, mediante chiamate ricorsive alla funzione stessa, **ciascuno dei due sottoarray** (passo **IMPERA** in DIVIDE-ET-IMPERA).
- Fa il **Merge dei due sottoarray** ordinati (passo **COMBINA** in DIVIDE-ET-IMPERA).



Merge Sort(A,p,q)

if $p < q$

then

$r \leftarrow \lfloor (p+q)/2 \rfloor$

Merge-Sort(A,p,r)

Merge-Sort(A,r+1,q)

Merge(A,p,q,r)

Merge

(R. Sedgewick, Algoritmi in C++, pp.353-360)

L' algoritmo di merging può essere realizzato in due modi:

- Merge a due vie
- Merge astratto sul posto

Merge (secondo Cormen)

- Il Cormen propone un Merge sul posto con sentinelle. Utilizza una procedura ausiliaria $\text{MERGE}(A, p, q, r)$ dove A è un array, p, q ed r sono indici tali che $p \leq q \leq r$.
- La procedura assume che i sottoarray $A[p, \dots q]$ e $A[q + 1, \dots r]$ siano ordinati.
- Ha complessità computazionale $\Theta(n)$, ove $n = r - p + 1$.

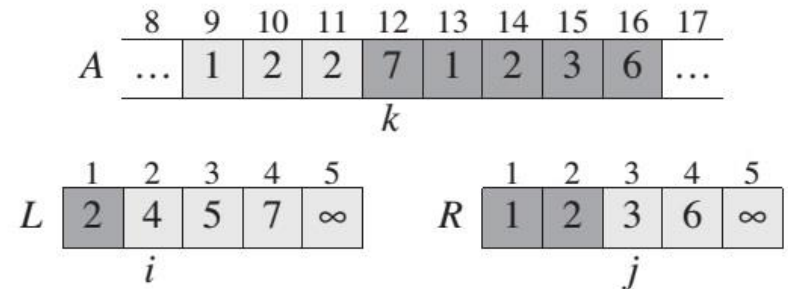
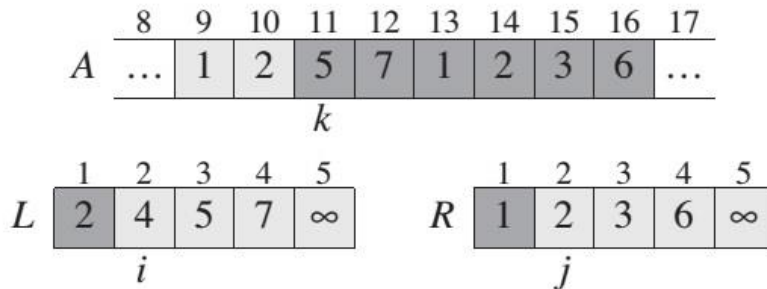
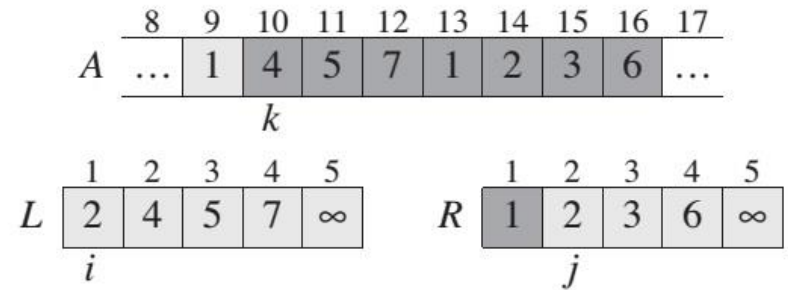
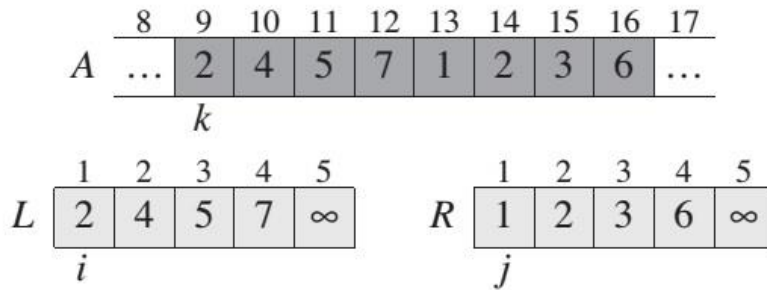
MERGE (A, p, q, r)

1. $n_1 \leftarrow q - p + 1$
2. $n_2 \leftarrow r - q$
3. Crea due nuovi array $L[1 \dots n_1 + 1]$ e $R[1 \dots n_2 + 1]$
4. for $i \leftarrow 1$ to n_1
5. $L[i] \leftarrow A[p + i - 1]$
6. for $j \leftarrow 1$ to n_2
7. $R[j] \leftarrow A[q + j]$
8. $L[n_1 + 1] \leftarrow +\infty$
9. $R[n_2 + 1] \leftarrow +\infty$
10. $i \leftarrow 1$
11. $j \leftarrow 1$
12. for $k \leftarrow p$ to r
13. if $L[i] \leq R[j]$
14. $A[k] \leftarrow L[i]$
15. $i \leftarrow i + 1$
16. else $A[k] \leftarrow R[j]$
17. $j \leftarrow j + 1$

MERGE: INVARIANTE DI CICLO

- All' inizio di ogni iterazione del ciclo for, il sottoarray $A[p \dots k - 1]$ contiene, ordinati, i $k - p$ elementi più piccoli di $L[1 \dots n_1 + 1]$ e $R[1 \dots n_2 + 1]$.
- Inoltre, $L[i]$ e $R[j]$ sono i più piccoli elementi dei loro array che non sono copiati in A .

Merge Sort



MERGE INVARIANTE DI CICLO: Inizializzazione

- Prima dell' iterazione di ogni ciclo, abbiamo $k = p$, quindi il sottoarray $A[p \dots k - 1]$ è vuoto e contiene i $k - p = 0$ elementi più piccoli di L e R .
- Inoltre, poichè $i = j = 1$ $L[i]$ e $R[j]$ sono i più piccoli elementi degli array che non sono stati ancora copiati in A .

MERGE INVARIANTE DI CICLO: Conservazione

- Supponiamo che $L[i] \leq R[j]$, quindi $L[i]$ è l'elemento più piccolo che non è stato ancora copiato in A . Poichè $A[p \dots k-1]$ contiene i $k-p$ elementi più piccoli, dopo aver copiato $L[i]$ in $A[k]$ il sottoarray $A[p \dots k]$ conterrà i $k-p+1$ elementi più piccoli. Incrementando k ed i si ristabilisce l'invariante di ciclo per la prossima iterazione.
- Se è $L[i] > R[j]$ il ragionamento è simile al precedente, sono le righe 16-17 che permettono di conservare l'invarianza di ciclo.

MERGE INVARIANTE DI CICLO:

Conclusione

- Alla fine del ciclo $k = r + 1$. Per l'invariante di ciclo il sottoarray $A[p \dots k - 1]$ contiene i $k - p = r - p + 1$ elementi ordinati che sono più piccoli di L e di R . Insieme gli array contengono $r - p + 3$ elementi. Tutti gli elementi tranne i due più grandi, le sentinelle, sono stati copiati in A .

Merge: Complessità

- Per verificare che la procedura MERGE viene eseguita in $\Theta(n)$ basta osservare che le righe 1-3 e 8-11 impiegano un tempo costante ed i cicli (righe 4-7) for impiegano un tempo $\Theta(n_1 + n_2) = \Theta(n)$, e ci sono n iterazioni del ciclo for (righe 12-17).

Merge Sort(A,p,q)

if $p < q$

then

$r \leftarrow \lfloor (p+q)/2 \rfloor$

Merge-Sort(A,p,r)

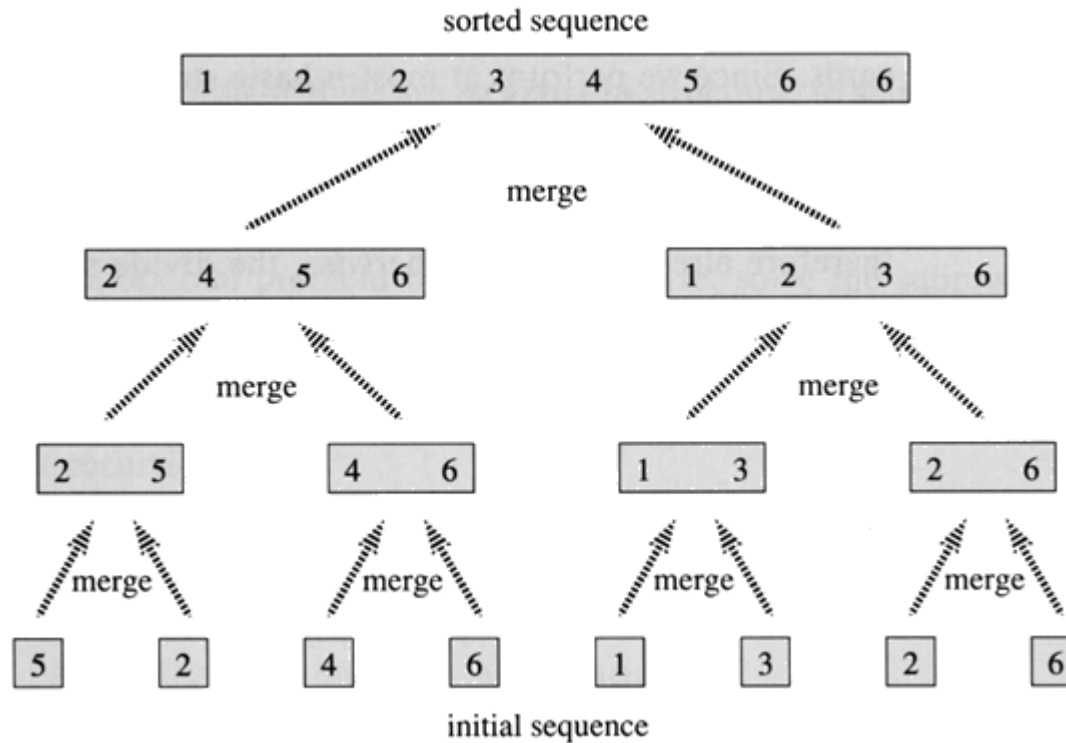
Merge-Sort(A,r+1,q)

Merge(A,p,q,r)

Merge Sort

- La procedura MERGE SORT ordina gli elementi nel sottoarray $A[p..r]$. Se $p \geq r$ il sottoarray ha al più un elemento; altrimenti il passo divide calcola semplicemente un indice q che separa l'array $A[p \dots r]$ in due sottoarray $A[p \dots q]$ e $A[q + 1, r]$ che contengono rispettivamente $\left\lfloor \frac{n}{2} \right\rfloor$ e $\left\lceil \frac{n}{2} \right\rceil$ elementi.

Merge Sort



Analisi degli algoritmi

Divide-et-Impera

- Quando un algoritmo contiene una chiamata ricorsiva a se stesso, il suo tempo di esecuzione può essere descritto con una **equazione di ricorrenza** o **ricorrenza**, che esprime il tempo di esecuzione totale di un problema di dimensione n in funzione del tempo di esecuzione per input più piccoli.
- Si utilizzano gli strumenti matematici per risolvere l'equazione di ricorrenza e stabilire i limiti delle prestazioni dell'algoritmo.

Ricorrenze degli algoritmi

Divide-et-Impera

- Una **ricorrenza** per il tempo di esecuzione di un algoritmo divide et impera si basa sui tre passi del paradigma di programmazione.
- Se la dimensione del problema è sufficientemente piccolo, ad esempio $n \geq c$, per qualche costante c , la soluzione richiede un tempo costante, che indichiamo con $\Theta(1)$.

(cont.)

- Supponiamo che la suddivisione generi sottoproblemi e che la dimensione di ciascun sottoproblema sia $\frac{1}{b}$ volte la dimensione del problema originale. È necessario $T\left(\frac{n}{b}\right)$ per risolvere un sottoproblema di dimensione $\frac{n}{b}$ e quindi, per risolverne a , $aT\left(\frac{n}{b}\right)$ tempo. Impieghiamo un tempo $D(n)$ per dividere il problema in sottoproblemi.

(cont.)

- Ed è necessario un tempo $C(n)$ per combinare le soluzioni dei sottoproblemi nella soluzione del problema originale.
- Pertanto si ottiene la ricorrenza:

$$\begin{cases} \Theta(1) & \text{se } n \leq c \\ aT\left(\frac{n}{b}\right) + D(n) + C(n) & \text{altrimenti} \end{cases}$$

Analisi di Merge Sort

- Nel caso del Merge Sort l'analisi si semplifica. Ogni passo divide genera due sottosequenze di dimensione pari a $\frac{n}{2}$.
- L'algoritmo merge sort applicato ad un solo elemento impiega un tempo costante $\Theta(1)$.
- Se abbiamo $n > 1$ elementi, suddividiamo il tempo di esecuzione nel modo seguente.

Analisi di Merge Sort

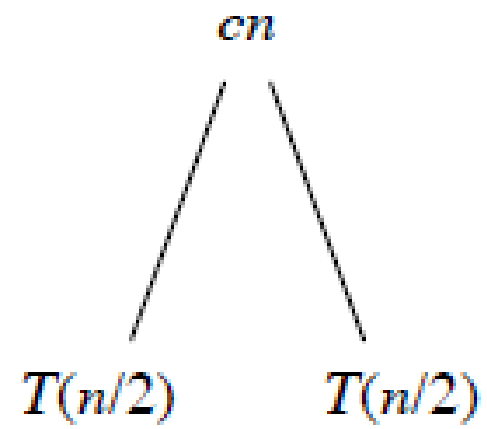
- Dividi
 - Questo passo calcola semplicemente il centro del sottoarray, richiedendo un tempo $\Theta(1)$.
- Impera
 - Risolviamo in modo ricorsivo i due sottoproblemi di dimensione $\frac{n}{2}$, con tempo totale di esecuzione.
- Combina
 - La procedura MERGE richiede un tempo $C(n) = \Theta(n)$

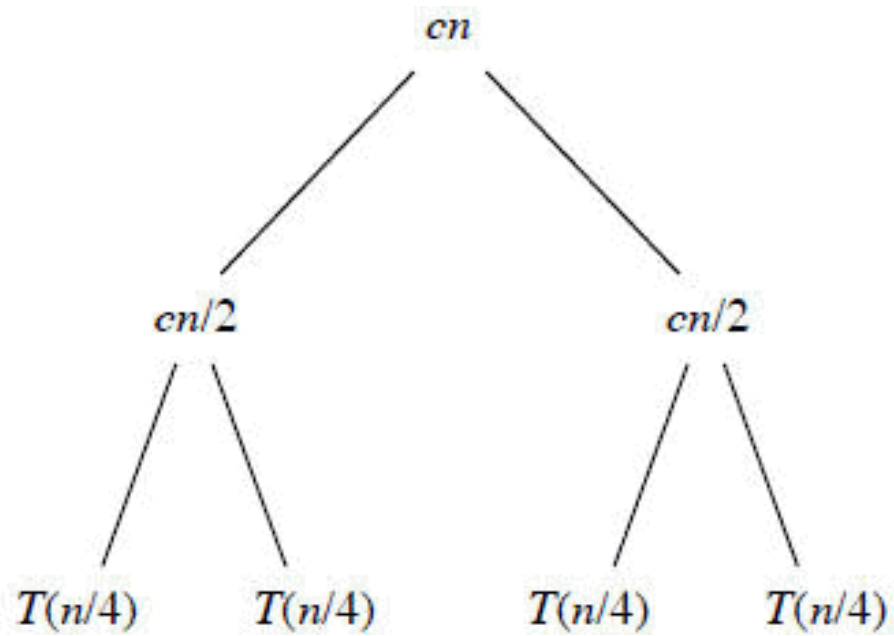
(cont.)

- Quando sommiamo le funzioni $D(n)$ e $C(n)$ stiamo sommando una funzione che è $\Theta(1)$ con una che è $\Theta(n)$, ottenendo ancora $\Theta(n)$.
- Sommandola al passo impera si ottiene la ricorrenza:
$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & \text{se } n > 1 \end{cases}$$

(cont.)

- Per comodità supponiamo che n sia Potenza esatta di 2. La parte a della figura mostra $T(n)$ che è stato espanso, nella parte b della figura, in un albero equivalente che mostra la ricorrenza. Il termine cn è la radice ed i due sottoalberi sono le ricorrenze $T\left(\frac{n}{2}\right)$. Il costo sostenuto è $c \frac{n}{2}$.
- Continuiamo ad espandere i nodi, finchè le dimensioni dei problemi si riducono ad 1, ciascuno con costo c .





Merge Sort : Albero di Ricorsione

